

Algorithms

Chapter 13 Red-Black Trees

可控制高度的二元樹

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **Properties of red-black trees**
- ▶ Rotations
- ▶ Insertion
- ▶ Deletion

binary search tree 所需時間為 $O(h)$

$h = \begin{cases} n & \text{最差, 跟 linked list 一樣} \\ \lg n & \text{最好} \end{cases}$

Overview

- ▶ A binary search tree of height h can implement any of the basic dynamic-set operations such as [SEARCH, PREDECESSOR, SUCCESSOR, MINIMUM, MAXIMUM, INSERT, and DELETE] in $O(h)$ time.

不会动到樹本身

- ▶ Thus, the set operations are fast if the height of the search tree is small; but if its height is large, their performance may be no better than with a linked list.

▶ Red-black trees

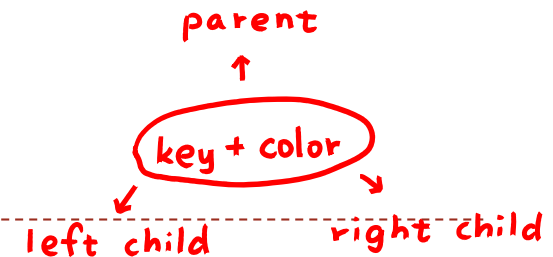
- ▶ A variation of binary search trees.
- ▶ **Balanced**: height is $O(\lg n)$, where n is the number of nodes.
- ▶ Operations will take $O(\lg n)$ time in the worst case.

紅黑樹: binary search tree + color attribute

可控制高度在 $\lg n$ 的常數倍

用顏色來控制高度

Properties of red-black trees_{1/2}



RB Tree's 性質

非黑即紅
root 必黑
葉子必黑
紅·紅不相連
黑高要一致
(怎麼走都)

▶ A **red-black tree** = a binary search tree + 1 bit per node: an attribute **color**, which is either red or black.

- ▶ Each node of the tree now contains the fields **color**, **key**, **left**, **right**, and **p**.
- ▶ If a child or the parent of a node does not exist, the corresponding pointer field of the node contains the value NIL.

▶ Red-black properties 性質

1. Every node is either red or black. 每節不是紅就是黑

上下都是黑 (2. The root is black. 根節點必為黑)

3. Every leaf (Nil) is black. 葉子必為黑

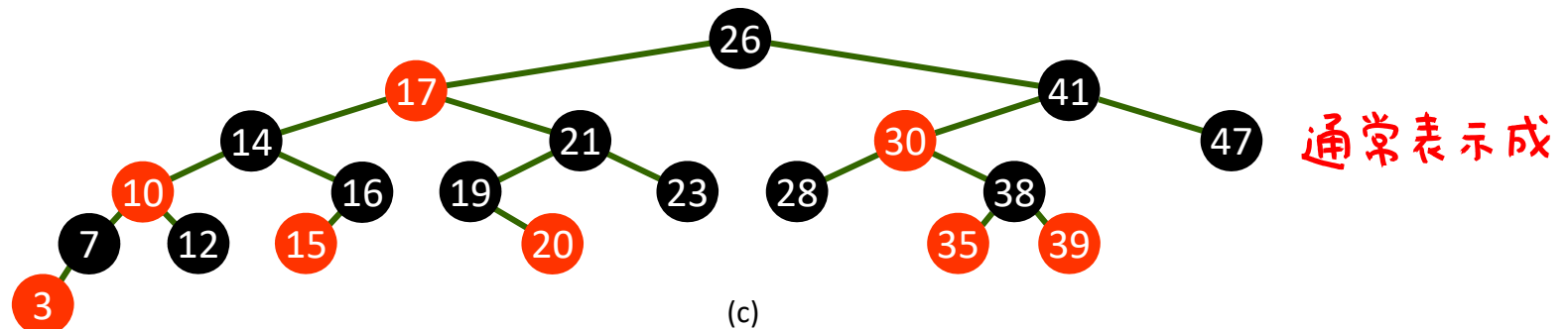
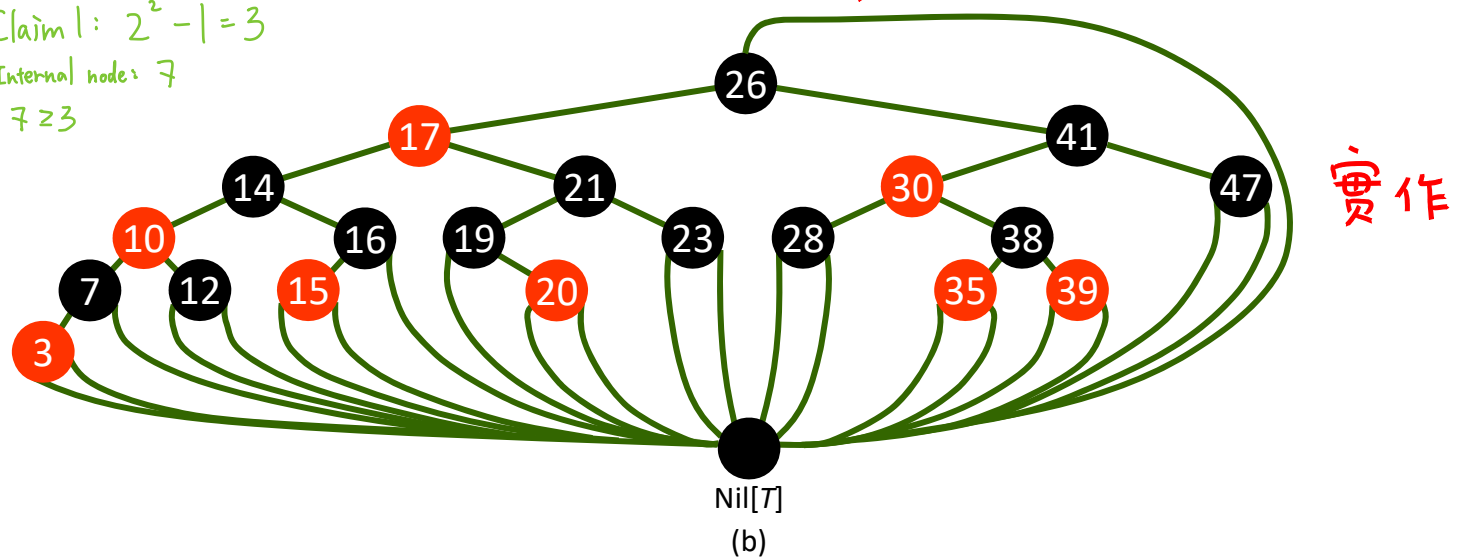
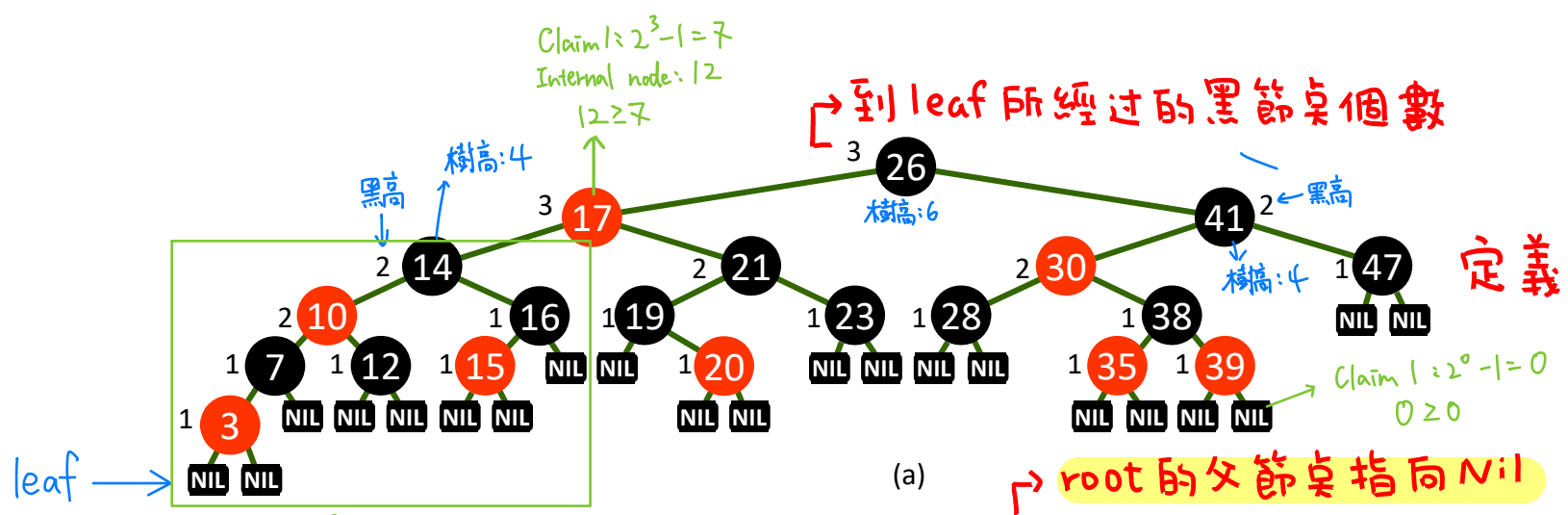
若自己為紅 ⇒ 兩子節點必為黑
(紅紅不能相連)

4. If a node is red, then both its children are black.

5. For each node, all paths from the node to descendant leaves contain

the same number of black nodes. (黑高要一樣) (從自己到任何一個 leaf 所經過的黑點都一致)

每一節到不同 leaf 的過程中, 所經過的黑節點個數皆相同



下面還有 NIL !! 不違反 leaf 是黑的規則

Properties of red-black trees_{2/2}

- ▶ All leaves are empty (nil) and colored black.
 - ▶ We use a single sentinel, $\text{nil}[T]$, for all the leaves as a matter of convenience. 用一個空 $\text{nil}[T]$ 代表所有 leaf
 - ▶ $\text{color}[\text{nil}[T]]$ is black. $\text{nil}[T]$ 是黑的
 - ▶ The root's parent is also $\text{nil}[T]$. root 的父節點也是 $\text{nil}[T]$
- ▶ **Height of a red-black tree**
 - ▶ **Height of a node** is the number of edges in a longest path to a leaf.
 - ▶ **Black-height** of a node x : $\text{bh}(x)$ is the number of black nodes (including $\text{nil}[T]$) on the path from x to leaf, not counting x . By property 5, black-height is well defined.

紅黑樹的高度 [node 高度: node 到 leaf 所經過 最多 edge 數 (數樹高的時候記得算 Nil 那個 leaf)
node 黑高: node 到 leaf 所經過 黑節點個數

The height of a red-black tree_{1/2}

- ▶ **Claim 1** The subtree rooted at any node x contains $\geq 2^{bh(x)} - 1$ internal nodes. 以 x 為 root 的子樹的 internal node 個數 $\geq 2^{bh(x)} - 1$

Proof: By induction on the height of x .

- ▶ **Basis:** 用樹高作歸納; 樹高比 x 低的都成立
強歸納

- ▶ Height of $x = 0 \Rightarrow x$ is a leaf $\Rightarrow bh(x) = 0$. $0 \geq 0$, 成立
- ▶ The subtree rooted at x has 0 internal nodes. $2^0 - 1 = 0$.

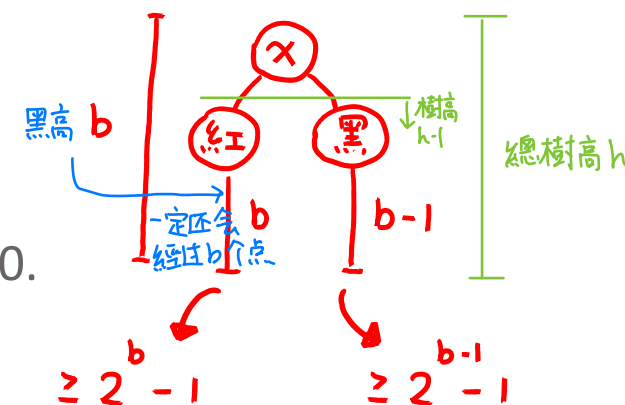
- ▶ **Inductive step:**

- ▶ Let $bh(x) = b$. 設黑高為 b
- ▶ Any child of x has black-height either b (if the child is red) or $b - 1$ (if the child is black). 若小孩為紅 $\Rightarrow$ 黑高為 b ; 若小孩為黑 $\Rightarrow$ 黑高為 $b - 1$

- ▶ By the inductive hypothesis, each child has $\geq 2^{bh(x)-1} - 1$ internal nodes.

- ▶ Thus, the subtree rooted at x contains $\geq 2 \cdot (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$ internal nodes. (The +1 is for x itself.)
2兒子的 internal node 數之和必超過 $b-1$ 加上自己

最差情形: 兩子節點皆為黑 $\Rightarrow$ 子節點 internal node $\geq 2^{b-1} - 1$



The height of a red-black tree_{2/2}

- ▶ **Lemma 1** A red-black tree with n internal nodes has height $\leq 2 \lg(n+1)$. 紅黑樹高度 $\leq \lg n$ 的常數倍 $\Rightarrow$ 想辦法控制樹的高度

Proof: h : 樹高 b : 黑高

- ▶ Let h and b be the height and black-height of the root, respectively.
- ▶ By Claim 1, we have $n \geq 2^b - 1$. 點數 $= 2^{\text{黑高}} - 1$, 由 Claim 1 $\Rightarrow$ 想辦法證點數與樹高的關係
 $\hookrightarrow$ 先把點數和黑高扯上關係
- ▶ By property 4, $\leq h/2$ nodes on the path from the node to a leaf are red. 紅紅不相連 $\leftarrow$ root 到 leaf 所經過的紅節點個數 $\leq \frac{h}{2}$ (紅色點最多是樹高的一半)
- ▶ Hence $\geq h/2$ are black, i.e., $b \geq h/2$. 黑節點個數 $\geq \frac{h}{2} \rightarrow$ 讓 black node 和樹高扯上關係
- ▶ Thus, $n \geq 2^b - 1 \geq 2^{h/2} - 1$.
- ▶ This implies $h \leq 2 \lg(n+1)$.

證明 rb tree 可以被實作
且操作為 $O(\lg n)$ 時間內可被完成

Operations on red-black trees

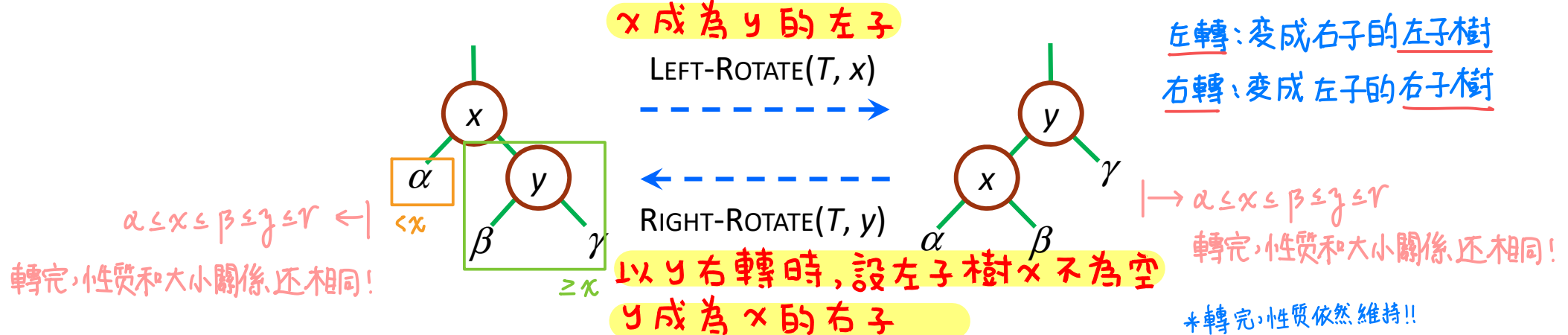
- ▶ The non-modifying binary-search-tree operations MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR, and SEARCH run in $O(\text{height})$ time. Thus, they take $O(\lg n)$ time on red-black trees.
- ▶ **Insertion and deletion** are not so easy. → 會改變樹的性質
- ▶ For example:
 - ▶ If we insert, what color to make the new node?
 - ▶ Red? Might violate property 4. 插入節點為紅 → 可能違反性質 4 (紅紅不相連)
 - ▶ Black? Might violate property 5. 插入節點為黑 → 可能違反性質 5 (黑高要一致)

Outline

- ▶ Properties of red-black trees
- ▶ **Rotations** ✱
- ▶ Insertion
- ▶ Deletion

Rotations 旋轉: 用來維持紅黑樹性質

- ▶ A local operation in a search tree that preserves the binary-search-tree property. 轉完後二元樹性質仍維持
(左子樹 < 自己, 右子樹 ≥ 自己)
- ▶ There are two kinds of rotations: 兩種旋轉方式
 - ▶ **Left rotations** and **right rotations**. 旋轉 = 左轉 + 右轉
 - ▶ They are inverses of each other. 左轉 + 右轉 = 沒有轉
- ▶ When we do a left rotation on a node x , we assume that its right child y is not $\text{nil}[T]$. 以 x 左轉時, 設右子樹 y 不為空



LEFT-ROTATE pseudocode

LEFT-ROTATE(T, x)

1. $y \leftarrow \text{right}[x]$
2. $\text{right}[x] \leftarrow \text{left}[y]$
3. **if** $\text{left}[y] \neq \text{nil}[T]$
4. $p[\text{left}[y]] \leftarrow x$
5. $p[y] \leftarrow p[x]$
6. **if** $p[x] = \text{nil}[T]$
7. $\text{root}[T] \leftarrow y$
8. **else if** $x = \text{left}[p[x]]$
9. $\text{left}[p[x]] \leftarrow y$
10. **else** $\text{right}[p[x]] \leftarrow y$
11. $\text{left}[y] \leftarrow x$
12. $p[x] \leftarrow y$

設定 x 與新右子
的關係 ①

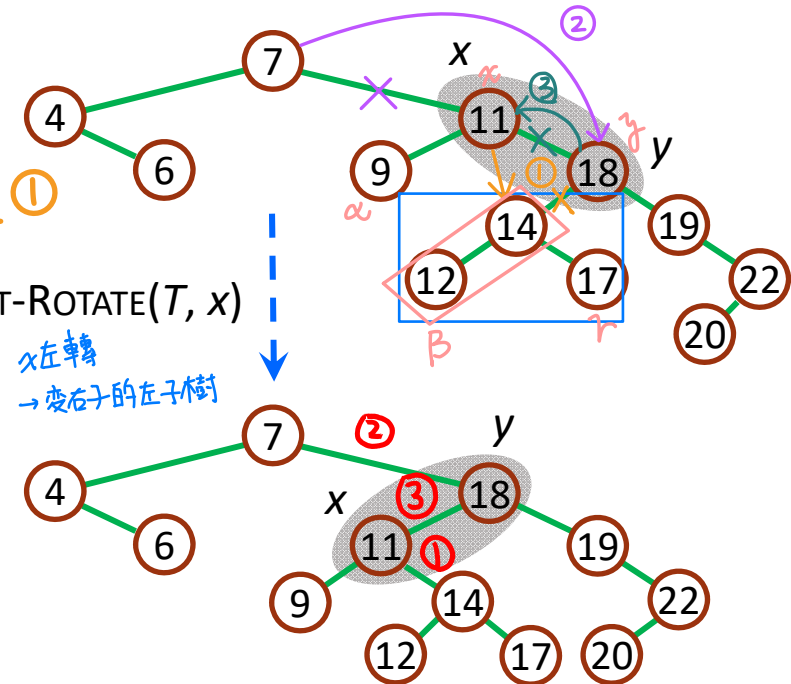
設定 y 與
新父親的
關係 ②

設定 x 與 y 的關係 ③

Left Rotate Step:
① $x \rightarrow \beta, y \nrightarrow \beta$
② x 的 parent $\rightarrow y$
 $x \nrightarrow x$ 的 parent
③ $y \rightarrow x, x \nrightarrow y$
(重設 x, y 親屬關係)

Right Rotate Step:
① $y \rightarrow \beta, x \nrightarrow \beta$
② y 的 parent $\rightarrow x$
 $y \nrightarrow y$ 的 parent
③ $x \rightarrow y, y \nrightarrow x$
(重設 x, y 親屬關係)

LEFT-ROTATE(T, x)



- ▶ Time: $O(1)$. 旋轉只需常數時間
- ▶ The code for RIGHT-ROTATE is symmetric.

Outline

- ▶ Properties of red-black trees
- ▶ Rotations
- ▶ **Insertion**
- ▶ Deletion

RB-Insertion_{1/2} 插入

- ▶ Start by doing regular binary-search-tree insertion.

RB-INSERT(T, z)

1. $y \leftarrow \text{NIL}; x \leftarrow \text{root}[T]$

2. **while** $x \neq \text{NIL}$

3. $y \leftarrow x$

4. **if** $\text{key}[z] < \text{key}[x]$

5. $x \leftarrow \text{left}[x]$

6. **else** $x \leftarrow \text{right}[x]$

7. $p[z] \leftarrow y$

8. **if** $y = \text{NIL}$

9. $\text{root}[T] \leftarrow z$ /* Tree T was empty */

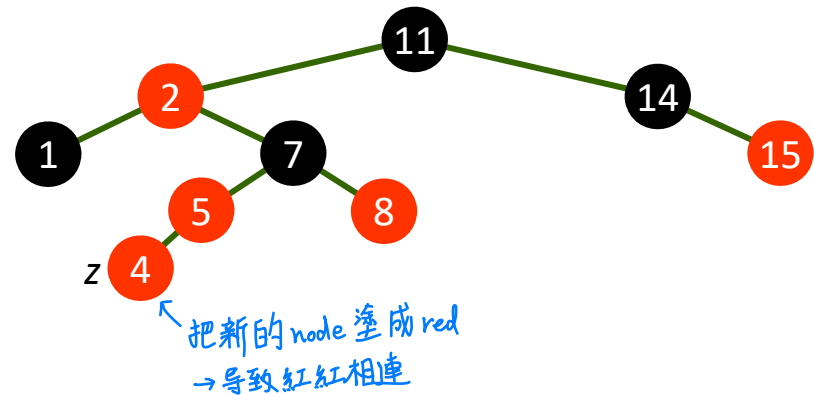
10. **else if** $\text{key}[z] < \text{key}[y]$

11. $\text{left}[y] \leftarrow z$

12. **else** $\text{right}[y] \leftarrow z$

13. $\text{left}[z] \leftarrow \text{nil}[T]; \text{right}[z] \leftarrow \text{nil}[T]; \text{color}[z] \leftarrow \text{RED}$ 設屬性

14. RB-INSERT-FIXUP(T, z) 修正



預設先塗紅色等等再修正

與之前相同
(二元搜尋樹)

RB-Insertion_{2/2}

- ▶ RB-INSERT ends by coloring the new node z red. 將插入的節塗成紅色
- ▶ Then it calls RB-INSERT-FIXUP because we could have violated a red-black property. 用 Fixup 修正
- ▶ Which property might be violated? 插入違反哪些性質
 1. OK.
 2. **If z is the root**, then there's a violation. Otherwise, OK. → 如果插入在第一個 (root) 就會違反
如果 z 是 root 則違反
 3. OK.
 4. **If $p[z]$ is red**, there's a violation: both z and $p[z]$ are red. → 有可能紅紅相連
如果 z 的父親是紅色則違反
 5. OK.

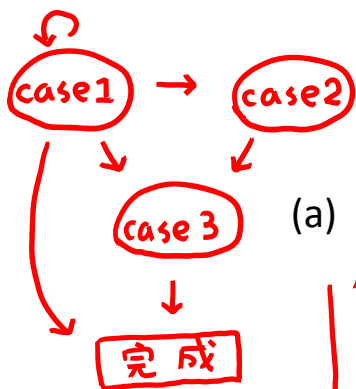
RB-INSERT-FIXUP procedure 修正

RB-INSERT-FIXUP(T, z)

1. **while** $color[p[z]] = \text{RED}$
2. **if** $p[z] = \text{left}[p[p[z]]]$
3. $y \leftarrow \text{right}[p[p[z]]]$
4. **if** $color[y] = \text{RED}$
5. $color[p[z]] \leftarrow \text{BLACK}$ Case 1
6. $color[y] \leftarrow \text{BLACK}$ Case 1
7. $color[p[p[z]]] \leftarrow \text{RED}$ Case 1
8. $z \leftarrow p[p[z]]$ Case 1
9. **else** {
10. **if** $z = \text{right}[p[z]]$ Case 2
11. $z \leftarrow p[z]$ Case 2
12. LEFT-ROTATE(T, z) Case 2
13. }
14. $color[p[z]] \leftarrow \text{BLACK}$ Case 3
15. $color[p[p[z]]] \leftarrow \text{RED}$ Case 3
16. RIGHT-ROTATE($T, p[p[z]]$) Case 3
17. **else** (same as **then** clause
18. with “right” and “left” exchanged)
19. $color[\text{root}[T]] \leftarrow \text{BLACK}$

只考慮父親是左子

(左子、右子實際上對稱)



(a)

往上兩層

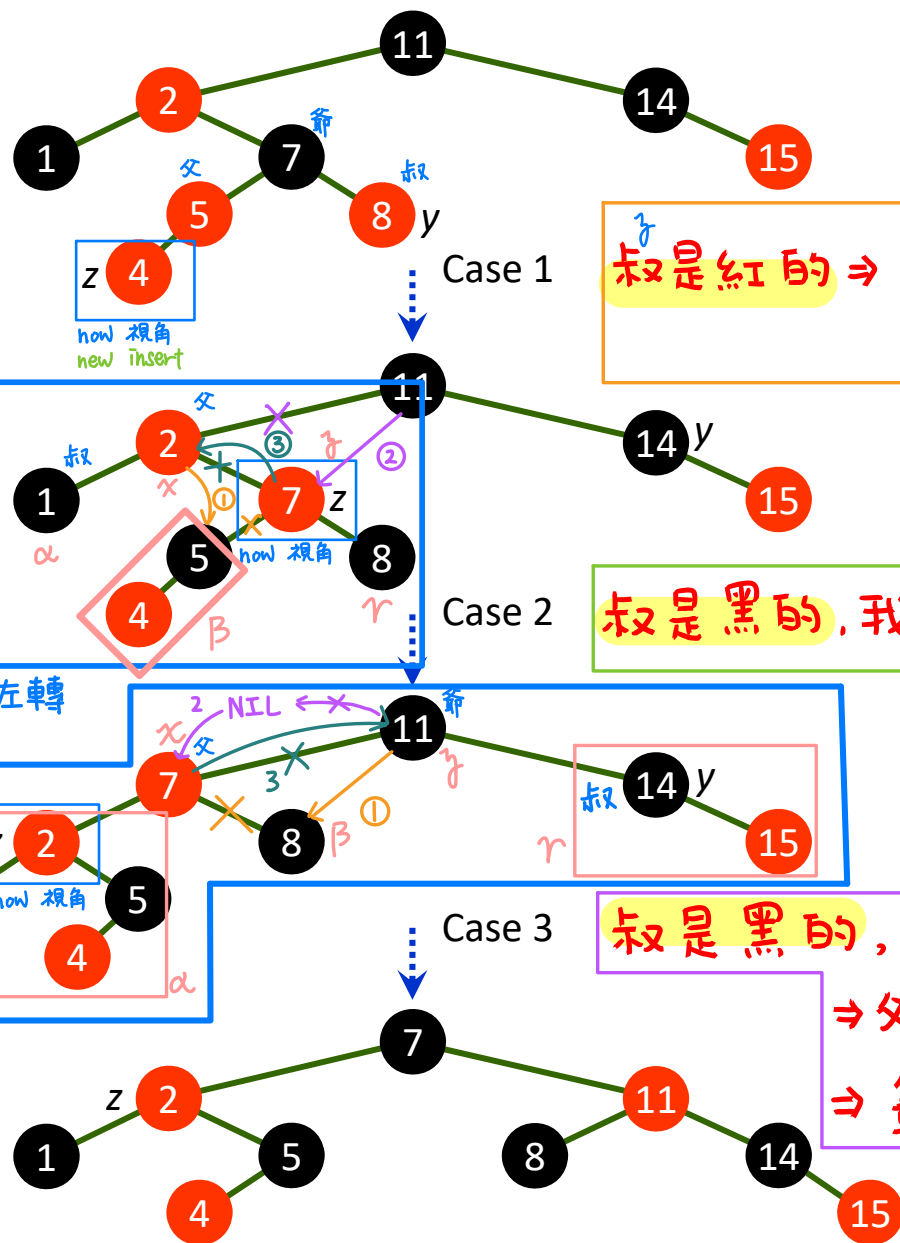
(b)

成為左子

(c)

完成

只考慮父親是左子



Case 1

Case 1

叔是紅的 $\Rightarrow$ 父、叔塗黑
爺塗紅

$\Rightarrow$ Case 1 完
一定會往上 2 層!

Case 2

Case 2

叔是黑的, 我是右子 $\Rightarrow$ 父左轉

Case 3

Case 3

叔是黑的, 我是左子
 $\Rightarrow$ 父塗黑, 爺塗紅
 $\Rightarrow$ 爺右轉

注意要維持“黑高要一樣”的性質

Left Rotate Step:

① $x \rightarrow \beta, y \rightarrow \beta$

② x 的 parent $\rightarrow y$

$x \rightarrow x$ 的 parent

③ $y \rightarrow x, x \rightarrow y$

(重設 x, y 親屬關係)

Right Rotate Step:

① $y \rightarrow \beta, x \rightarrow \beta$

② y 的 parent $\rightarrow x$

$y \rightarrow y$ 的 parent

③ $x \rightarrow y, y \rightarrow x$

(重設 x, y 親屬關係)

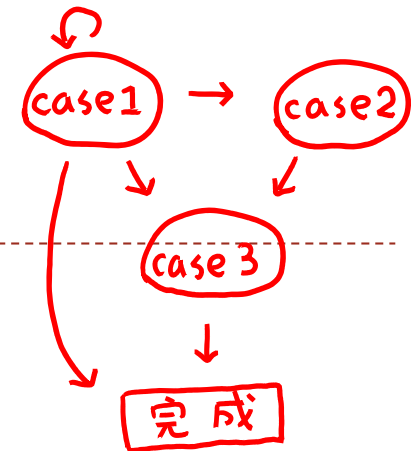
正確性證法: loop invariant 要維持的性質

每一次 while loop 開始之前具有下列性質

Correctness of RB-INSERT

- ▶ **Loop invariant:** At the start of each iteration of the **while** loop of lines 1-16,
 - a. Node z is red. z 是紅的 如果 z 的父親是 root, 則 $p[z]$ 是黑色的
 - b. If $p[z]$ is the root, then $p[z]$ is black. 
 - c. There is at most one red-black violation: 最多違反性質 2 或性質 4
 - ▶ Property 2, z is the root and is red. 證明修正後黑高一樣
 - ▶ Property 4, both z and $p[z]$ are red.
- ▶ We omit the further details for proving the correctness.

Time complexity of RB-INSERT



► Analysis:

- Each iteration takes $O(1)$ time.
- The **while** loop repeats only if case 1 is executed, and then the pointer z moves two levels up the tree. **case 1 每次往上 2 個 level**
- The **while** loop terminates if case 2 or case 3 is executed.
- $O(\lg n)$ levels $\rightarrow O(\lg n)$ time. $\rightarrow$ 最多就是 Case 1 一直做, 做 $O(\lg n)$ 次
- Also note that there are at most 2 rotations overall.

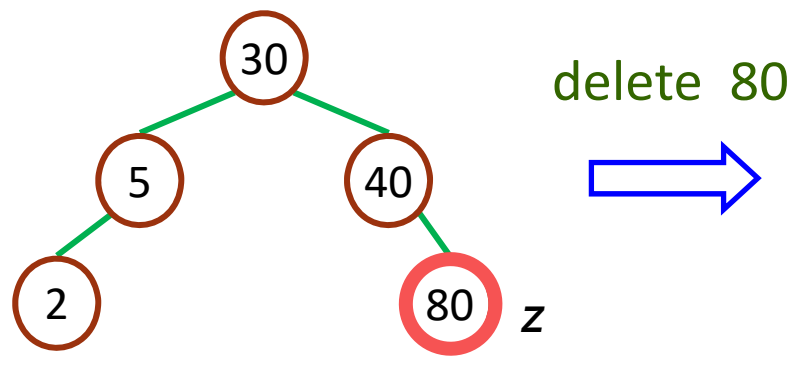
最多轉 2 次 $\begin{cases} \text{case 2 左轉 1 次} \\ \text{case 3 右轉 1 次} \end{cases}$

- Thus, insertion into a red-black tree takes $O(\lg n)$ time.

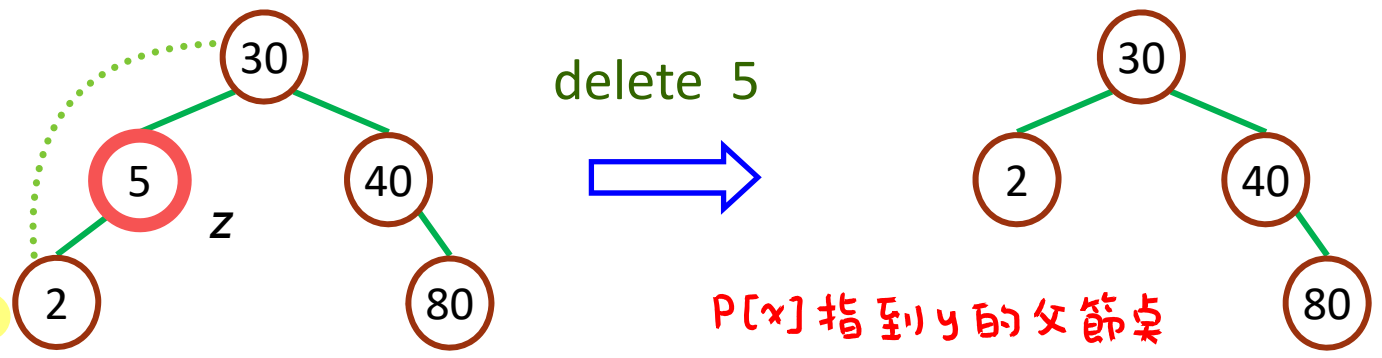
Outline

- ▶ Properties of red-black trees
- ▶ Rotations
- ▶ Insertion
- ▶ **Deletion**

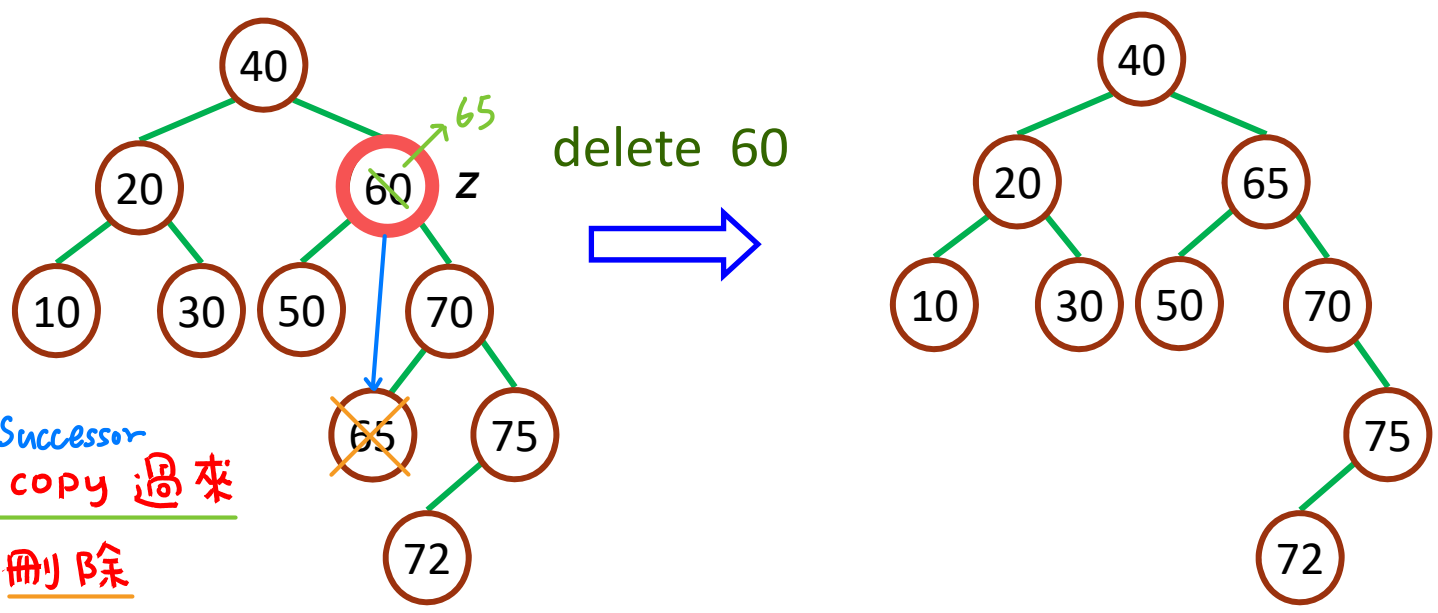
Case 1:
沒有兒子，
直接刪



Case 2:
有一個兒子，
將兒子連到
(配的)父親



Case 3:
有2個兒子，
① 找下一個 Successor
② 將下一個 copy 過來
③ 將下一個刪除



RB-Deletion_{1/3}

y是真正要刪的桌

x 是 $\begin{cases} y \text{ 的唯一-子节点} \\ nil[T] \end{cases}$

1. if $left[z] = \text{NIL}$ or $right[z] = \text{NIL}$

```
3. else  $y \leftarrow \text{Tree-Successor}(z)$ 
```

5. $x \leftarrow \text{left}[y]$

7. $p[x] \leftarrow p[y]$

9. $root[T] \leftarrow x$

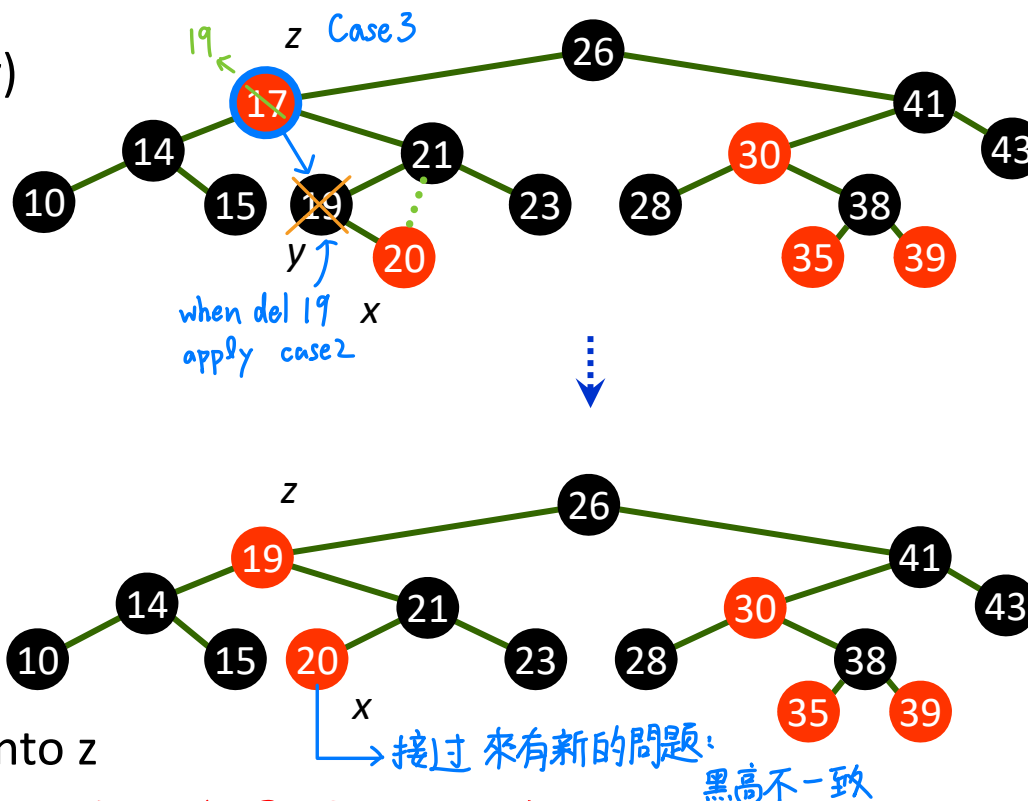
11. $left[p[y]] \leftarrow x$

13. **if** $y \neq z$

15. copy y 's satellite data into z

17. RB-DELETE-FIXUP(T, x)

```
18. return y
```



刪掉的真是黑的⇒修正

y 是真正要删的桌

→ 刪紅色不需要修正!

RB-Deletion_{2/3}

- ▶ y is the node that was actually spliced out. y 是真正要刪的節點
- ▶ x is either
 - x 是 $\begin{cases} y \text{ 的唯一子節點} \\ \text{nil}[T] \end{cases}$
 - ▶ y 's sole non-sentinel child before y was spliced out, or
 - ▶ the sentinel, if y had no children.
- ▶ In both cases, $p[x]$ is now the node that was previously y 's parent.
 $p[x]$ 指到 y 的父節點
- ▶ If y is red, the red-black properties still hold when y is spliced out, for the following reasons: 若 y 是紅的, 刪除不違反紅黑樹性質
 - ▶ no black-heights in the tree have changed, 黑高不改變
 - ▶ no red nodes have been made adjacent, and 紅節點不相連
 - ▶ since y could not have been the root if it was red, the root remains black. $\therefore y$ 非 root $\therefore$ root 仍為黑

RB-Deletion_{3/3} 若 y 是黑的, 删除可能違反紅黑樹性質

► If y is black, we could have violations of red-black properties:

1. OK.

如果 y 是 root 且 x 是紅色則違反

2. If y is the root and x is red, then the root has become red.

3. OK.

4. Violation if $p[y]$ and x are both red. 會違反如果 $p[y]$ 和 x 都是紅色

5. Any path containing y now has 1 fewer black node.

path 會經過 y 的桌黑高都少 1

► Correct this problem by giving x an “extra black”. 讓 x 多黑色屬性

► Now property 5 is OK, but property 1 is not.

► x is either **doubly black** or **red & black**.

x 變成黑黑 或 x 變成紅黑

↳ 把這個點丟上 root

↳ 直接紅變黑, 問題解決!

把 y 刪掉後,

讓 y 的屬性由 x 繼承

若 y 是黑色, 刪掉時讓 x 繼承 y 的屬性, 則:

x $\begin{cases} \text{紅} \rightarrow \text{紅黑} \\ \text{黑} \rightarrow \text{黑黑} \end{cases} \rightarrow$ 這樣就可以正常計算黑高了!

→ 讓 node 多繼承上層的顏色屬性可以解決「黑高不一致」的問題
卻會違反性質 1: node color 非黑即紅

RB-DELETE-FIXUP

x 是左子的 4 Case

兄是紅 ①
兄是黑 { 右姪黑 { 左姪黑 ②
 { 右姪紅 ④ { 左姪紅 ③

- ▶ **Idea:** Move the extra black up the tree until 將多的黑色往上移, 直到
 - 1. x points to a red & black node $\rightarrow$ turn it into a black node, 遇到紅父 Case 1 遇到紅塗黑
 - 2. x points to the root $\rightarrow$ just remove the extra black, or 遇到 root Case 2
 - 3. suitable rotations and recolorings can be performed.
- ▶ 過程中可以適當的旋轉和將父的顏色重塗
- ▶ Within the while loop:
 - x 指到 doubly black node 且非 root
 - ▶ x always points to a nonroot doubly black node. ↗
 - ▶ w is x 's sibling. w 是 x 的兄弟
 - ▶ w cannot be $nil[T]$, since that would violate property 5 at $p[x]$.
 w 不會是 leaf, 否則違反性質 5 (黑高要一樣)
- ▶ There are 8 cases, 4 of which are symmetric to the other 4. 共 8 種情形
- ▶ As with insertion, the cases are not mutually exclusive.
We'll look at cases in which x is a left child. 只考慮 x 在左子的情形

4 種

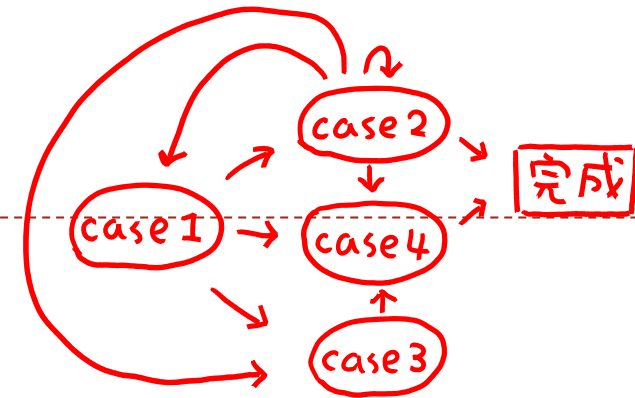
RB-DELETE-FIXUP procedure

RB-DELETE-FIXUP(T, x)

```

1.  while  $x \neq \text{root}[T]$  and  $\text{color}[x] = \text{BLACK}$ 
2.    if  $x = \text{left}[p[x]] \Rightarrow$  只考虑  $x$  是左子
3.       $w \leftarrow \text{right}[p[x]]$ 
4.      if  $\text{color}[w] = \text{RED}$ 
5.         $\text{color}[w] \leftarrow \text{BLACK}$ 
6.         $\text{color}[p[x]] \leftarrow \text{RED}$ 
7.        LEFT-ROTATE( $T, p[x]$ )
8.         $w \leftarrow \text{right}[p[x]]$ 
9.        if  $\text{color}[\text{left}[w]] = \text{BLACK}$  and  $\text{color}[\text{right}[w]] = \text{BLACK}$ 
10.          $\text{color}[w] \leftarrow \text{RED}$ 
11.          $x \leftarrow p[x]$ 
12.        else if  $\text{color}[\text{right}[w]] = \text{BLACK}$ 
13.           $\text{color}[\text{left}[w]] \leftarrow \text{BLACK}$ 
14.           $\text{color}[w] \leftarrow \text{RED}$ 
15.          RIGHT-ROTATE( $T, w$ )
16.           $w \leftarrow \text{right}[p[x]]$ 
17.           $\text{color}[w] \leftarrow \text{color}[p[x]]$ 
18.           $\text{color}[p[x]] \leftarrow \text{BLACK}$ 
19.           $\text{color}[\text{right}[w]] \leftarrow \text{BLACK}$ 
20.          LEFT-ROTATE( $T, p[x]$ )
21.           $x \leftarrow \text{root}[T]$ 
22.        else (same as then clause with "right" and "left" exchanged)
23.           $\text{color}[x] \leftarrow \text{BLACK}$ 

```

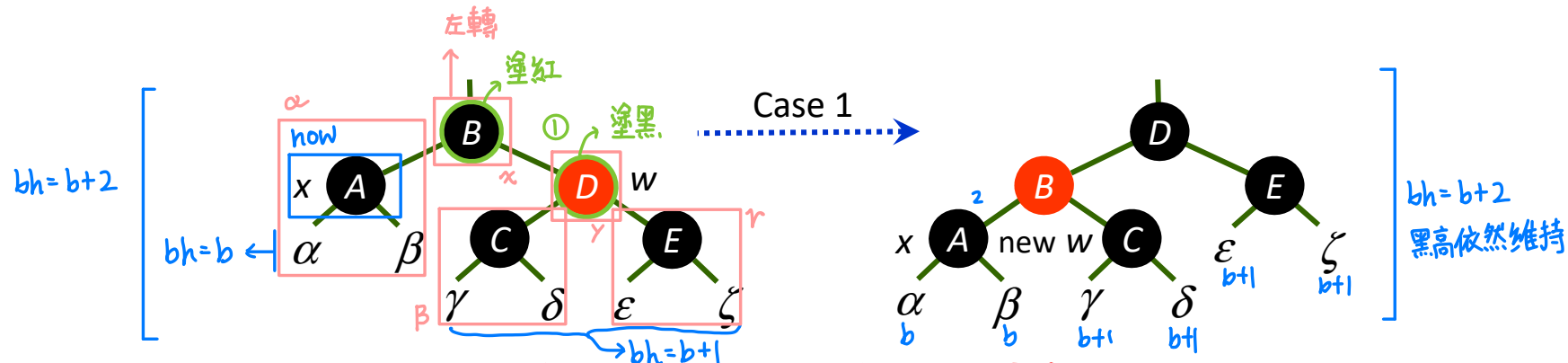


Case 1 * Case 1 $\rightarrow$ Case 2 $\rightarrow$ 必完成
 Case 1 不会反覆 Case 1、Case 2 横跳
 Case 1 * Case 2 每次做都会上升一层
 Case 1 Worst Case: Case 2 一直做, 做 $O(\lg n)$ 次
 Case 2
 Case 2
 Case 3
 Case 3
 Case 3
 Case 3
 Case 4
 Case 4
 Case 4
 Case 4
 Case 4

注意要維持“黑高要一樣”的性質

Case 1: w is red 只是紅

x 有“2 黑高”



假設黑高(α) = 黑高(β) = b 則黑高(γ) = 黑高(δ) = 黑高(ϵ) = 黑高(ζ) = $b+1$

- ▶ w must have black children. ① 兄塗黑，父塗紅
- ▶ Make w black and $p[x]$ red. ② 父左轉
- ▶ Then left rotate on $p[x]$. ③ 兄立刻成為黑 → 到 Case 2、3、4
- ▶ New sibling of x was a child of w before rotation → must be black.
- ▶ Go immediately to case 2, 3, or 4.

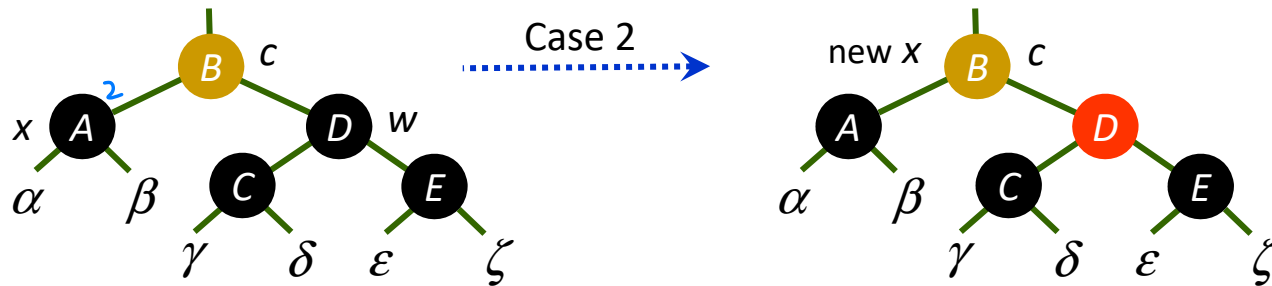
Case 1 做完後爸一定變紅

兄的兒子

兄是黑, 兩個姪子也是黑

Case 2: w is black & both of w 's children are black

±黃: 可能是黑 or 紅



黑高(α) = 黑高(β) = 黑高(γ) = 黑高(δ) = 黑高(ϵ) = 黑高(ζ) = b

x, w 少一黑
相當 c 加一黑

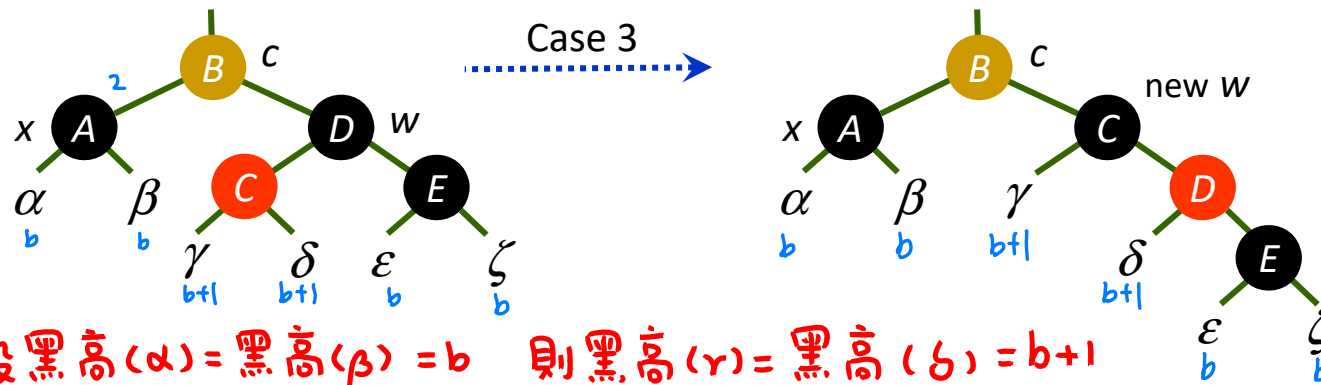
- ▶ Take 1 black off x ($\rightarrow$ singly black) and off w ($\rightarrow$ red). ① 我和兄的黑移轉一個給父親 (兩個兒子少一黑的情況下, 爸才能變黑)
 - ▶ Move that black to $p[x]$.
 - ▶ Do the next iteration with $p[x]$ as the new x . ② 父成為新的 x
 - ▶ If entered this case from case 1, then $p[x]$ was red $\rightarrow$ new x is red & black $\rightarrow$ color attribute of new x is RED $\rightarrow$ loop terminates. ③ 父如果是紅 $\rightarrow$ 結束
- Then new x is made black in the last line.

Case 1 $\rightarrow$ Case 2
紅黑 $\rightarrow$ 把他當黑

如果是 case 1 過來的 $\rightarrow$ 父為紅 $\rightarrow$ 結束

w is black, 兄是黑, 兄的左兒左姪是紅, 兄的右兒右姪是黑

Case 3: w 's left child is red, and w 's right child is black



假設黑高(α) = 黑高(β) = b 則黑高(γ) = 黑高(δ) = $b+1$

黑高(ϵ) = 黑高(ζ) = b

- ▶ Make w red and w 's left child black.
- ▶ Then right rotate on w .
- ▶ New sibling w of x is black with a red right child $\rightarrow$ case 4.

① 兄塗紅, 左姪塗黑

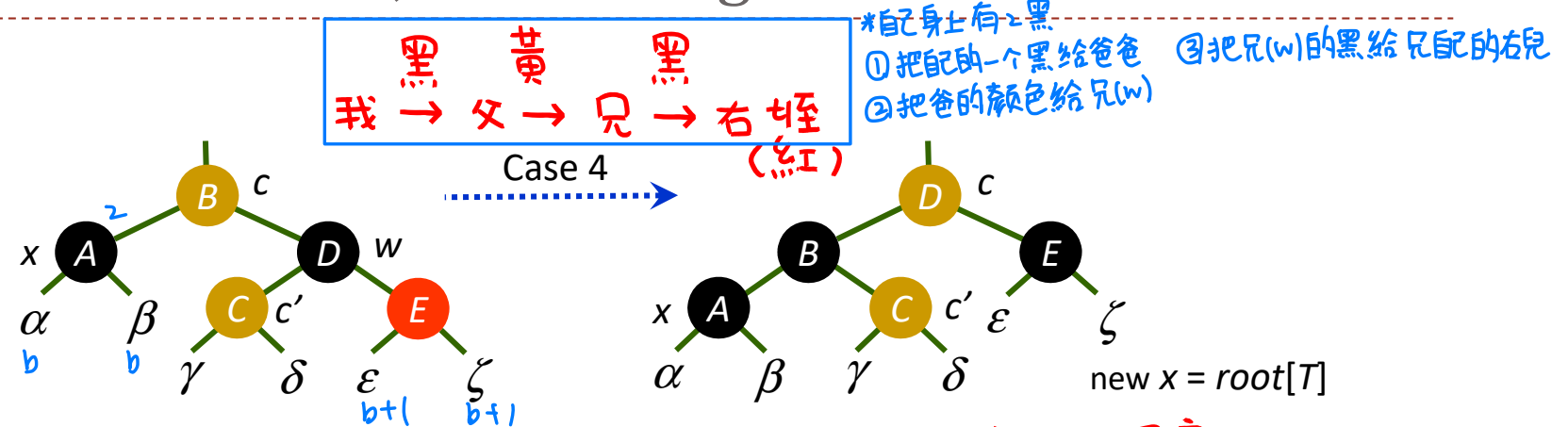
② 兄右轉

③ 成為 case 4

兄是黑, 右姪是紅

注意要維持"黑高要一樣"的性質

Case 4: w is black, and w 's right child is red



假設黑高(α) = 黑高(β) = b

C 的颜色 { 紅 $\Rightarrow$ 黑高(r) = 黑高(b) = $b+1$
黑 $\Rightarrow$ 黑高(r) = 黑高(b) = b

- ▶ Make w be $p[x]$'s color (c). 黑高(ϵ) = 黑高(ζ) = $b+1$
- ▶ Make $p[x]$ black and w 's right child black.
- ▶ Then left rotate on $p[x]$.
- ▶ Remove the extra black on x ($\Rightarrow$ x is now singly black) without violating any red-black properties. $\Rightarrow$ 完成
- ▶ All done. Setting x to root causes the loop to terminate.

- ① w 塗成父的颜色
- ② 父塗黑, 右姪塗黑
- ③ 父左轉
- ④ 移除 x 的一個黑

$\Rightarrow$ 做完後
沒有人有2黑

Time complexity of RB-DELETE

► Analysis:

- Case 2 is the only case in which more iterations occur. *case 2 会重複做*
 - x moves up 1 level. *每次上升 1 level*
 - Hence, $O(\lg n)$ iterations.

- Each of cases 1, 3, and 4 has 1 rotation $\Rightarrow \leq 3$ rotations in all.
最多轉 3 次

- Thus, the overall time for RB-DELETE is therefore $O(\lg n)$.

- <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>
(Red-black Tree Animation)

*如果是 case 1 $\Rightarrow$ case 2 $\Rightarrow$ 結束
case 1 跳 case 2, 必結束*

